FarmaRAG — Assistente de Bulas com IA Local

11 de outubro de 2026 · Sérgio

Visão geral e objetivo

O FarmaRAG é um assistente para o balcão da farmácia. Ele responde perguntas sobre medicamentos usando apenas as bulas oficiais e sempre mostra o trecho e a seção de onde tirou a resposta.

Tudo roda localmente, com Ollama, sem API paga e sem nuvem.

Objetivo de portfólio: mostrar que você sabe construir um RAG de ponta a ponta, medir a qualidade dele com um benchmark próprio e entregar um backend Python completo, com dataset em JSON e interface em JavaScript.

Meta do MVP: 30 a 50 medicamentos comuns indexados, respostas com citação em menos de 30 segundos num notebook com 16 GB de RAM.

Problema, público e casos de uso

No balcão, as dúvidas sobre medicamentos precisam de resposta rápida, mas a bula é longa e o PDF é difícil de pesquisar. O FarmaRAG encontra o trecho certo e resume com a fonte ao lado.

Público: farmacêutico e atendente de farmácia. O paciente não é usuário direto do sistema.

Caso de uso	Pergunta de exemplo	Seção da bula que responde
Interação	Pode tomar ibuprofeno junto com losartana?	Interações medicamentosas
Contraindicação	Gestante pode usar este medicamento?	Contraindicações / Advertências
Posologia	Qual a dose usual para adulto?	Posologia e modo de usar
Conservação	Precisa ficar na geladeira depois de aberto?	Cuidados de armazenamento
Reação adversa	Sonolência é um efeito comum?	Reações adversas

Escopo do MVP

O MVP faz uma coisa bem feita: perguntar sobre bulas e receber resposta citada. Todo o resto fica para depois.

Entra no MVP:

- Ingestão de PDFs de bulas (versão para o profissional de saúde)
- Busca semântica por trechos, com filtro opcional por medicamento
- Resposta gerada pelo LLM local, sempre com citação (medicamento, seção, página)
- Resposta "não encontrei nas bulas" quando o contexto não basta
- Tela simples de pergunta e resposta, com histórico
- Benchmark próprio com métricas de qualidade

Fica fora do MVP:

- Leitura de receita por foto (OCR)
- Recomendação personalizada para um paciente
- Controle de estoque ou integração com sistemas de farmácia
- Fine-tuning de modelo
- Login com perfis e permissões complexas

Arquitetura em blocos (para desenhar no draw.io)

O sistema tem seis blocos, com o backend em Python e o frontend em JavaScript, divididos em duas partes: ingestão (roda uma vez, offline) e consulta (roda a cada pergunta). Esta seção lista as peças e as ligações; o desenho fica com você.

Bloco	Responsabilidade	Tecnologia
Frontend	Tela de pergunta, resposta, fontes e histórico	React + Vite (JavaScript)
API	Recebe a pergunta, salva histórico, busca trechos, monta o prompt, chama o LLM, confere a citação	FastAPI + Pydantic
Ollama	Roda o LLM e o modelo de embeddings	Ollama (nativo na máquina)
Banco relacional	Histórico de perguntas e metadados das bulas	PostgreSQL + SQLAlchemy
Banco vetorial	Trechos das bulas e seus embeddings	pgvector (no mesmo PostgreSQL)
Pipeline de ingestão	PDF → bula em JSON → trechos em JSONL → embeddings → banco	Script Python (pdfplumber)

Ligações (setas) para desenhar:

1. Frontend → API (REST)
2. API → Banco relacional (salva e lê histórico)
3. API → Ollama (embedding da pergunta e geração da resposta)
4. API → Banco vetorial (busca dos trechos mais parecidos)
5. Pipeline de ingestão → Ollama (embeddings dos trechos)
6. Pipeline de ingestão → Banco vetorial (grava trechos + metadados)

Dicas de desenho: use duas raias horizontais, "Ingestão" e "Consulta". Desenhe uma moldura em volta do que vai no Docker Compose. Deixe o Ollama fora da moldura, porque ele roda nativo para usar a GPU.

Perguntas para responder no desenho: por que o frontend chama a API em vez de falar direto com o Ollama e o banco? E dentro do bloco da API, em quais camadas você separaria o código (rotas, serviço de RAG, acesso ao banco)? Desenhe essas camadas como caixas dentro do bloco.

Fluxos principais

São dois fluxos: a ingestão prepara as bulas uma vez, e a consulta responde cada pergunta usando o que a ingestão guardou.

Fluxo de ingestão (offline):

1. Baixar manualmente os PDFs das bulas do Bulário Eletrônico da ANVISA e salvar em `data/raw/`
2. Extrair o texto de cada PDF, página por página
3. Limpar o texto, separar por seção e salvar a bula processada em `data/processed/bulas/<id>.json`
4. Dividir cada seção em trechos, sem misturar seções, e salvar todos em `data/processed/chunks.jsonl`
5. Gerar o embedding de cada trecho pelo Ollama
6. Carregar trechos e embeddings no pgvector. Os JSONs continuam sendo a fonte da verdade: o banco pode ser apagado e recriado a partir deles

Fluxo de consulta (online):

1. O usuário digita a pergunta no frontend
2. O frontend envia a pergunta para a API FastAPI
3. A API valida a pergunta (Pydantic) e registra no histórico
4. A API gera o embedding da pergunta pelo Ollama
5. Busca os trechos mais parecidos no banco vetorial (filtrando por medicamento, se informado)
6. Monta o prompt: instruções + trechos encontrados + pergunta
7. O LLM gera a resposta, citando os trechos usados
8. A API confere se as citações apontam para trechos que realmente vieram na busca
9. A API salva a resposta no histórico e devolve resposta + fontes para o frontend

Pergunta para pensar: no passo 4, o que acontece se o usuário perguntar sobre um medicamento que não está indexado? Como o sistema deveria perceber isso antes de chamar o LLM?

Dataset em JSON

O dataset tem três arquivos: um JSON por bula processada, um JSONL com todos os trechos e um JSON com as perguntas do benchmark. Eles são a fonte da verdade e vão versionados no Git; os PDFs originais e o banco, não.

1. Bula processada — `data/processed/bulas/<id>.json`, um arquivo por bula:

```
{
  "id": "losartana-potassica-50mg-fabricante-x",
  "medicamento": "Losartana potássica 50 mg",
  "principio_ativo": "losartana potássica",
  "fabricante": "Fabricante X",
  "tipo_bula": "profissional",
  "data_versao": "2025-03-15",
  "fonte_pdf": "data/raw/losartana_fabricante_x.pdf",
  "processado_em": "2026-10-12T10:00:00-03:00",
  "secoes": [
    {
      "secao": "interacoes",
      "titulo_original": "6. INTERAÇÕES MEDICAMENTOSAS",
      "pagina_inicio": 4,
      "pagina_fim": 5,
      "texto": "..."
    }
  ]
}
```


O campo `secao` usa sempre uma destas chaves fixas, para que a busca e o benchmark possam filtrar por seção: `indicacoes`, `contraindicacoes`, `advertencias`, `interacoes`, `armazenamento`, `posologia`, `reacoes_adversas`, `superdose`, `outras`. O `titulo_original` guarda o título como aparece no PDF, porque cada fabricante escreve de um jeito.

2. Trechos — `data/processed/chunks.jsonl`, um objeto JSON por linha:


```
{"chunk_id": "losartana-potassica-50mg-fabricante-x__interacoes__003",
 "bula_id": "losartana-potassica-50mg-fabricante-x",
 "medicamento": "Losartana potássica 50 mg", "secao": "interacoes",
 "pagina": 4, "ordem": 3, "texto": "...", "n_tokens": 287}
```


O `chunk_id` segue o padrão `<bula_id>__<secao>__<ordem com 3 dígitos>`. Assim ele é estável: gerar os trechos de novo com os mesmos parâmetros produz os mesmos ids, e o benchmark continua válido.

3. Benchmark — `data/benchmark/perguntas.json`, uma lista de perguntas:


```
[
  {
    "id": "q001",
    "pergunta": "Quais medicamentos interagem com a losartana segundo a bula?",
    "respondivel": true,
    "bula_id": "losartana-potassica-50mg-fabricante-x",
    "secao_esperada": "interacoes",
    "chunk_ids_esperados": ["losartana-potassica-50mg-fabricante-x__interacoes__003"],
    "resposta_esperada": "...",
    "categoria": "interacao"
  },
  {
    "id": "q041",
    "pergunta": "Qual o preço médio deste medicamento?",
    "respondivel": false,
    "bula_id": null,
    "secao_esperada": null,
    "chunk_ids_esperados": [],
    "resposta_esperada": null,
    "categoria": "fora_do_escopo"
  }
]
```


4. Resultados do benchmark — `data/benchmark/resultados/<data>_<modelo>.json`, gerado pelo script, com a configuração usada (modelo, top_k, tamanho do trecho), as métricas agregadas e o resultado de cada pergunta. Guardar a configuração junto é o que permite comparar rodadas depois.

Pergunta para pensar: se a bula de um medicamento for atualizada pela ANVISA, o que muda nos três arquivos? Quais perguntas do benchmark podem quebrar?

Banco de dados

O PostgreSQL guarda quatro tabelas: duas espelham o dataset JSON (`bulas`, `chunks`) e duas registram o uso (`consultas`, `consulta_fontes`). As duas primeiras são recriadas pelo script `carregar_banco.py` sempre que o dataset mudar.

Tabela `bulas`

Coluna	Tipo	Observação
id	text, chave primária	Igual ao `id` do JSON
medicamento	text	Nome exibido na tela
principio_ativo	text	Usado no seletor do frontend
fabricante	text	
data_versao	date	Mostrada junto da resposta
fonte_pdf	text	Caminho do PDF original

Tabela `chunks`

Coluna	Tipo	Observação
chunk_id	text, chave primária	Igual ao `chunk_id` do JSONL
bula_id	text, FK → bulas.id	Índice para filtrar por medicamento
secao	text	Índice para filtrar por seção
pagina	integer	
ordem	integer	Posição do trecho dentro da seção
texto	text	
embedding	vector(N)	N = dimensão do modelo de embeddings (bge-m3 usa 1024)

Crie um índice HNSW na coluna `embedding` com distância de cosseno. Com poucos milhares de trechos a busca funciona sem índice; vale medir a diferença.

Tabela `consultas`

Coluna	Tipo	Observação
id	uuid, chave primária	Gerado pela API
pergunta	text	Sem dados de paciente (ver Segurança)
bula_id_filtro	text, pode ser nulo	Medicamento escolhido no seletor
resposta	text, pode ser nulo	Nulo quando deu erro
status	text	`respondida`, `nao_encontrada` ou `erro`
modelo	text	Nome do LLM usado
latencia_ms	integer	Tempo total da consulta
criado_em	timestamptz	

Tabela `consulta_fontes`

Coluna	Tipo	Observação
consulta_id	uuid, FK → consultas.id	
chunk_id	text, FK → chunks.chunk_id	
posicao	integer	Número da citação na resposta ([1], [2]...)
score	real	Similaridade retornada pela busca
citado	boolean	Se o LLM realmente usou esse trecho

Pergunta para pensar: se `chunks` for recriada e um `chunk_id` antigo sumir, o que acontece com a FK em

`consulta_fontes`? Você prefere bloquear, apagar em cascata ou guardar uma cópia do texto na consulta?

Contratos da API

A API tem cinco endpoints, todos em JSON, sob o prefixo `/api/v1`. Defina cada corpo como um modelo Pydantic; o FastAPI gera a documentação em `/docs` automaticamente, e ela vira o contrato com o frontend.

Método	Rota	Para que serve
GET	`/api/v1/saude`	Diz se a API, o banco e o Ollama estão de pé
GET	`/api/v1/medicamentos`	Lista as bulas indexadas, para o seletor
POST	`/api/v1/perguntas`	Recebe uma pergunta e devolve resposta + fontes
GET	`/api/v1/historico?limite=20&pagina=1`	Lista as consultas mais recentes
GET	`/api/v1/historico/{consulta_id}`	Mostra uma consulta completa com as fontes

Requisição de `POST /api/v1/perguntas`:

```
{
  "pergunta": "A losartana interage com anti-inflamatórios?",
  "bula_id": "losartana-potassica-50mg-fabricante-x"
}
```

Regras de validação: `pergunta` com 5 a 500 caracteres, sem espaços sobrando; `bula_id` opcional, mas se vier precisa existir.

Resposta de sucesso (200):

```
{
  "consulta_id": "6f1c2e4a-...",
  "status": "respondida",
  "resposta": "Segundo a bula, ... [1]. Além disso, ... [2].",
  "fontes": [
    {
      "posicao": 1,
      "chunk_id": "losartana-potassica-50mg-fabricante-x__interacoes__003",
      "medicamento": "Losartana potássica 50 mg",
      "secao": "interacoes",
      "pagina": 4,
      "data_versao": "2025-03-15",
      "trecho": "...",
      "score": 0.82
    }
  ],
  "modelo": "nome-do-modelo:3b",
  "latencia_ms": 14250
}
```

Quando o sistema não encontra base nas bulas, a resposta também é 200, com `"status": "nao_encontrada"`, `"resposta": null` e `"fontes": []`. Não encontrar é um resultado válido, não um erro.

Formato de erro (todas as rotas):

```
{"erro": {"codigo": "OLLAMA_INDISPONIVEL",
          "mensagem": "O modelo não respondeu. Tente de novo em instantes."}}
```

HTTP	Código	Quando
404	`BULA_NAO_ENCONTRADA`	`bula_id` informado não existe
422	`ENTRADA_INVALIDA`	Falhou a validação do Pydantic
502	`CITACAO_INVALIDA`	O LLM citou trechos que não recebeu, mesmo após nova tentativa
503	`OLLAMA_INDISPONIVEL`	Ollama fora do ar
503	`BANCO_INDISPONIVEL`	PostgreSQL fora do ar
504	`TEMPO_ESGOTADO`	Geração passou do limite configurado

Libere CORS só para a origem do frontend (por padrão `http://localhost:5173`).

Prompt e parâmetros de geração

O prompt tem três partes (regras, trechos numerados e pergunta) e pede a saída em JSON, para que o código valide as citações sem precisar interpretar texto livre. Abaixo está a estrutura; a redação final das regras é sua, e o benchmark diz qual versão funciona melhor.

1. Regras (mensagem de sistema) — o que o modelo precisa obedecer:

- Responder em português, só com base nos trechos fornecidos
- Citar cada afirmação com o número do trecho, no formato [n]
- Se os trechos não respondem à pergunta, devolver `nao_encontrada` em vez de responder
- Não recomendar dose para um paciente específico; apenas relatar o que a bula diz
- Devolver somente o JSON pedido, sem texto em volta

2. Trechos (contexto) — cada trecho entra numerado, com cabeçalho de origem:

```
[1] Losartana potássica 50 mg | seção: interacoes | página 4
<texto do trecho>

[2] ...
```

3. Pergunta — o texto do usuário, depois dos trechos.

Saída esperada do modelo:

```
        {"status": "respondida", "resposta": "... [1] ... [2].", "citacoes": [1, 2]}
```

O Ollama permite forçar a saída em JSON, o que reduz respostas quebradas em modelos pequenos.

Validação depois da geração:

1. O JSON tem os campos esperados?
2. Todo número em `citacoes` existe entre os trechos enviados?
3. Todo [n] que aparece no texto está em `citacoes`, e vice-versa?
4. Se algo falhar, tentar uma vez de novo; se falhar outra vez, responder `CITACAO_INVALIDA`

Parâmetro	Valor inicial	Observação
Temperatura	0,1	Baixa, para respostas estáveis e repetíveis no benchmark
Trechos enviados (top_k)	5	Ajustar na etapa 2 pelo recall@k
Tamanho do trecho	~300 tokens	Com sobreposição de ~50 tokens
Janela de contexto (num_ctx)	4096 tokens	Cabe 5 trechos + regras + pergunta; janelas maiores gastam mais memória
Limite de resposta	400 tokens	Respostas de balcão são curtas
Limiar de similaridade	a definir	Abaixo dele, não chama o LLM e devolve `nao_encontrada`

Pergunta para pensar: como você vai escolher o valor do limiar de similaridade? Que perguntas do benchmark ajudam a encontrar esse número?

Frontend em JavaScript

O frontend é um app React com Vite, em JavaScript puro, com duas telas: Consulta e Histórico. Ele só fala com a API; nunca acessa o banco nem o Ollama.

Tela de Consulta:

- Seletor opcional de medicamento, carregado de `GET /api/v1/medicamentos`
- Campo de pergunta com contador de caracteres (máximo 500) e botão Perguntar
- Indicador de carregamento, porque a resposta pode levar de 10 a 30 segundos
- Resposta com os marcadores [1], [2] clicáveis: ao clicar, a fonte correspondente é destacada
- Lista de fontes, cada uma com medicamento, seção, página, data da bula e o trecho

Tela de Histórico:

- Lista paginada das consultas, com pergunta, status e data
- Ao clicar, abre a consulta completa com as fontes

Em todas as telas: aviso fixo no rodapé dizendo que o assistente não substitui o farmacêutico nem a bula.

Componente / arquivo	Responsabilidade
`App.jsx`	Navegação entre Consulta e Histórico
`api.js`	Todas as chamadas à API e o tratamento do formato de erro
`SeletorMedicamento.jsx`	Busca e mostra a lista de bulas
`FormPergunta.jsx`	Campo, validação local e envio
`RespostaCard.jsx`	Texto da resposta com citações clicáveis
`FonteCard.jsx`	Um trecho citado, com origem
`ListaHistorico.jsx`	Lista paginada de consultas
`AvisoLegal.jsx`	Rodapé fixo

Estados da tela de Consulta — cada um precisa de uma aparência própria:

Estado	Quando acontece	O que a tela mostra
Ocioso	Antes da primeira pergunta	Campo vazio e exemplos de perguntas
Carregando	Esperando a API	Indicador e botão desabilitado
Respondida	`status = respondida`	Resposta + fontes
Não encontrada	`status = nao_encontrada`	Mensagem clara, sem parecer erro, sugerindo escolher o medicamento
Erro	Resposta com `erro`	Mensagem do erro e botão Tentar de novo

Pergunta para pensar: a resposta chega como texto com "[1]". Como você transforma esses marcadores em elementos clicáveis sem usar `dangerouslySetInnerHTML`?

Stack e restrições de hardware

Com 16 GB de RAM e 2 GB de VRAM, o projeto cabe se o LLM for de 3 a 4B parâmetros, quantizado em Q4. Os valores de memória abaixo são aproximados; meça os reais na etapa 0.

Peça	Escolha sugerida	Memória aproximada
LLM	Modelo de 3–4B, Q4 (Qwen, Gemma ou Llama pequenos; o benchmark decide)	2,5–3,5 GB (parte na GPU, parte na RAM)
Embeddings	Modelo multilíngue (bge-m3 ou multilingual-e5), porque as bulas são em português	0,5–1,5 GB
Banco	PostgreSQL + pgvector, em Docker	~0,5 GB
API	FastAPI + Pydantic + SQLAlchemy, em Docker	~0,5 GB
Frontend	React + Vite (JavaScript), servidor de desenvolvimento em Node	~0,3 GB
Ingestão	pdfplumber (ou pypdf) + cliente Python do Ollama	roda só na ingestão
Testes	pytest para o código; script próprio para o benchmark	—

Somando tudo, o sistema usa entre 4 e 6 GB e sobra espaço para o sistema operacional e a IDE.

Restrições práticas:

- Rode o Ollama nativo, fora do Docker. GPU dentro de container no Windows exige WSL2 e configuração extra.
- Modelos de 7–8B podem entrar só no benchmark, como comparação, não no uso do dia a dia.
- Gere os embeddings das bulas uma vez só e guarde no banco; refazer isso a cada execução é lento.
- Confirme se a placa é NVIDIA. Se não for, planeje rodar tudo na CPU e preferir modelos de 1–3B.

Estrutura de pastas e configuração

O repositório tem cinco partes independentes: backend, ingestão, benchmark, dados e frontend. Ingestão e benchmark são scripts que você roda na mão; só o backend, o banco e o frontend ficam no ar.

```
farmarag/
├── backend/
│   ├── app/
│   │   ├── main.py          # cria o FastAPI, CORS, registra rotas
│   │   ├── config.py        # lê o .env
│   │   ├── api/             # rotas: perguntas, historico, medicamentos, saude
│   │   ├── schemas/         # modelos Pydantic de entrada e saída
│   │   ├── services/        # busca, montagem do prompt, validação de citações
│   │   ├── llm/             # cliente do Ollama (geração e embeddings)
│   │   └── db/              # modelos SQLAlchemy, sessão, repositórios
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
├── ingestao/
│   ├── extrair_pdf.py       # PDF → texto por página
│   ├── separar_secoes.py    # texto → bula em JSON
│   ├── gerar_chunks.py      # bulas → chunks.jsonl
│   └── carregar_banco.py    # chunks.jsonl + embeddings → pgvector
├── benchmark/
│   ├── rodar_benchmark.py   # perguntas.json → resultados/<data>_<modelo>.json
│   └── metricas.py
├── data/
│   ├── raw/                 # PDFs (fora do Git)
│   ├── processed/bulas/     # um JSON por bula
│   ├── processed/chunks.jsonl
│   └── benchmark/           # perguntas.json e resultados/
├── frontend/
│   ├── src/                 # App.jsx, api.js, components/
│   └── package.json
├── docker-compose.yml       # postgres+pgvector e backend
├── .env.example
└── README.md
```

Variáveis do `.env` — tudo que muda entre experimentos fica aqui, nunca fixo no código:

Variável	Exemplo	Para que serve
`OLLAMA_URL`	`http://host.docker.internal:11434`	Endereço do Ollama nativo, visto de dentro do container
`LLM_MODELO`	nome do modelo de 3–4B	Trocado a cada rodada do benchmark
`EMBED_MODELO`	`bge-m3`	Modelo de embeddings
`EMBED_DIM`	`1024`	Precisa bater com a coluna `vector(N)`
`DATABASE_URL`	`postgresql+psycopg://farmarag:senha@db:5432/farmarag`	Conexão do SQLAlchemy
`TOP_K`	`5`	Trechos enviados ao LLM
`LIMIAR_SIMILARIDADE`	a definir	Corte para `nao_encontrada`
`CHUNK_TOKENS` / `CHUNK_SOBREPOSICAO`	`300` / `50`	Usados pela ingestão
`TEMPERATURA`	`0.1`	Geração
`TIMEOUT_LLM_S`	`60`	Limite antes do erro `TEMPO_ESGOTADO`
`CORS_ORIGENS`	`http://localhost:5173`	Origem permitida do frontend

O `.env` real fica fora do Git; o `.env.example` vai no repositório com valores de exemplo.

Erros, timeouts e testes

Cada falha previsível tem um comportamento definido e um teste que o confirma. Os testes automáticos rodam sem Ollama; o benchmark, que usa o modelo de verdade, roda à parte.

Falha	Comportamento da API	O que o frontend mostra
Ollama fora do ar	503 `OLLAMA_INDISPONIVEL`, sem tentar de novo	"Modelo indisponível" + Tentar de novo
Geração demora mais que `TIMEOUT_LLM_S`	Cancela e devolve 504 `TEMPO_ESGOTADO`	Sugere uma pergunta mais curta ou específica
LLM devolve JSON quebrado	Tenta mais uma vez; depois 502	Mensagem genérica de erro
LLM cita trecho inexistente	Tenta mais uma vez; depois 502 `CITACAO_INVALIDA`	Mensagem genérica de erro
Nenhum trecho acima do limiar	200 com `nao_encontrada`, sem chamar o LLM	Estado Não encontrada
Banco fora do ar	503 `BANCO_INDISPONIVEL`	"Serviço indisponível"

Toda consulta, inclusive as que falham, entra na tabela `consultas` com o status certo. Assim dá para ver depois onde o sistema mais erra.

Testes automáticos (pytest):

Tipo	O que testar	Depende de
Unidade	`separar_secoes` reconhece títulos de seção escritos de formas diferentes	Textos de exemplo em `tests/fixtures/`
Unidade	`gerar_chunks` nunca mistura duas seções e gera `chunk_id` estável	Uma bula JSON pequena de exemplo
Unidade	Validação de citações aceita as válidas e rejeita as inventadas	Saídas de LLM escritas à mão
Contrato	Os JSONs do dataset seguem o formato definido	Um validador (Pydantic ou JSON Schema)
Integração	`POST /perguntas` devolve cada status e cada erro da tabela acima	Ollama substituído por um falso (mock) + banco de teste

Benchmark (manual): `rodar_benchmark.py` usa o Ollama real e grava o resultado em `data/benchmark/resultados/`. Rode sempre que mudar modelo, prompt, tamanho de trecho ou limiar.

Pergunta para pensar: por que vale a pena trocar o Ollama por um falso nos testes de integração, se o benchmark já testa o sistema completo?

Avaliação e benchmark

O benchmark é o que diferencia este projeto de um tutorial de RAG. Monte um conjunto de 50 perguntas com resposta

conhecida e meça cada etapa separadamente.

Conjunto de teste:

- 40 perguntas respondíveis, cada uma com a resposta esperada, o medicamento e a seção de origem
- 10 perguntas sem resposta nas bulas indexadas, para testar se o sistema sabe dizer "não encontrei"
- Escreva as perguntas antes de olhar o que o sistema responde, para não viciar o teste

Métrica	O que mede	Como calcular
Recall@k	Se o trecho certo veio entre os k primeiros da busca	Perguntas com o trecho certo no top-k ÷ total respondível
Fidelidade	Se a resposta está sustentada pelos trechos citados	Revisão manual (sim/não) por resposta
Citação correta	Se a citação aponta para a seção certa	Citações corretas ÷ total de respostas
Recusa correta	Se diz "não encontrei" quando deve	Recusas certas ÷ 10 perguntas sem resposta
Latência	Tempo da pergunta até a resposta	Mediana em segundos

Experimentos para comparar:

- 2 ou 3 LLMs pequenos diferentes (e um de 7–8B como referência)
- 2 tamanhos de trecho na ingestão
- Busca com e sem filtro por medicamento

Pergunta para pensar: se o recall@k estiver baixo, adianta trocar o LLM? Em qual bloco da arquitetura está o problema nesse caso?

Segurança, ética e limites

O sistema lida com saúde, então ele tem que preferir dizer "não sei" a inventar. Essas regras fazem parte do projeto e mostram maturidade para quem avaliar o portfólio.

- **Só responde com base nas bulas.** Se os trechos não sustentam a resposta, o sistema diz que não encontrou.
- **Sempre cita a fonte.** Toda resposta mostra medicamento, seção e página, para o profissional conferir.
- **Não personaliza para paciente.** Perguntas como "posso dar isso ao meu filho de 5 anos?" recebem o que a bula diz para a faixa etária e a orientação de consultar o farmacêutico ou médico.
- **Aviso fixo na tela:** o assistente não substitui o farmacêutico nem a leitura da bula.
- **LGPD:** o histórico guarda só a pergunta e a resposta, sem nome, CPF ou dados do paciente.
- **Bulas mudam:** guarde a data da versão de cada bula e mostre essa data na resposta.
- **É um projeto de portfólio.** Uso real em farmácia exigiria validação por farmacêuticos antes.

Roadmap por etapas

São oito etapas, da mais arriscada para a mais conhecida: primeiro a parte de IA, que é a novidade, depois a API e a interface, que são mais previsíveis. Cada etapa termina com algo que funciona e uma pergunta para você responder antes de seguir.

Etapa 0 — Ambiente. Instalar o Ollama, baixar um modelo de 3B e um de embeddings, medir tempo de resposta e uso de memória.
- Pergunta: quantos tokens por segundo você conseguiu? Isso cabe na meta de 30 segundos?

Etapa 1 — Ingestão de 5 bulas. Script que lê os PDFs, separa por seção e gera os trechos com metadados.
- Pergunta: as seções ficaram bem separadas? Onde a extração do PDF quebrou?

Etapa 2 — Busca, sem LLM. Gravar no pgvector e testar a busca com 10 perguntas, medindo o recall@k.
- Pergunta: qual valor de k e qual tamanho de trecho deram o melhor recall?

Etapa 3 — Geração com citação. Montar o prompt, chamar o LLM e conferir as citações, num módulo Python que ainda não depende da API.
- Pergunta: o modelo inventa citações? Como seu código detecta isso?

Etapa 4 — Benchmark completo. Indexar 30 a 50 bulas, rodar as 50 perguntas e comparar modelos.
- Pergunta: qual modelo você escolheria, e qual métrica pesou mais na decisão?

Etapa 5 — API principal. FastAPI com endpoints de pergunta e histórico, usando o PostgreSQL e o módulo de RAG da etapa 3.
- Pergunta: o que a API faz se o Ollama demorar ou cair?

Etapa 6 — Frontend. Tela em React de pergunta e resposta, com as fontes e o histórico.
- Pergunta: como mostrar as fontes de um jeito que o farmacêutico realmente confira?

Etapa 7 — Entrega. Docker Compose, README com o desenho da arquitetura e os resultados do benchmark, e um vídeo

curto de demonstração.

- Pergunta: alguém consegue rodar o projeto do zero só seguindo o seu README?